# Introduction à la Programmation et l'Algorithmique

CHRISTELLE CAILLOUET – CHRISTELLE.CAILLOUET@UNIV-COTEDAZUR.FR

DINO LOPEZ PACHECO – DINO.LOPEZ@UNIV-COTEDAZUR.FR

# Quoi faire avec des tableaux ?

- Rechercher des propriétés
  - Tableau trié ?
  - Valeur Min ? Max ? Moyenne ?
  - Rechercher une Valeur, son nombre d'occurences
  - Tableau vide ? Plein ?
- Faire des actions sur les tableaux
  - Trier
  - Insérer une Valeur
  - Supprimer une valeur

# Rechercher des propriétés

- Tableau trié, min, max, moyenne
  - Faut-il absolument parcourir tout le tableau ?
  - Non
    - Pour ce qui est de savoir si le tableau est trié
    - Dès qu'on a une valeurqui indique que le tableau n'est pas trié, on DOIT s'arrêter
  - Oui
    - Pour calculer le min, max, moyenne
    - Le min, max peuvent se trouver n'importe où, y compris dans la dernière case du tableau

# Rechercher une valeur

- On ne DOIT pas parcourir tout le tableau
- Algo de recherche Classique

```
int exempleRechercheValeur (int t[], int taille, int val)
{
    while ( (valeur non trouvée) && (pas à la fin du tableau) )
        On se déplace à l emplacement suivant du tableau

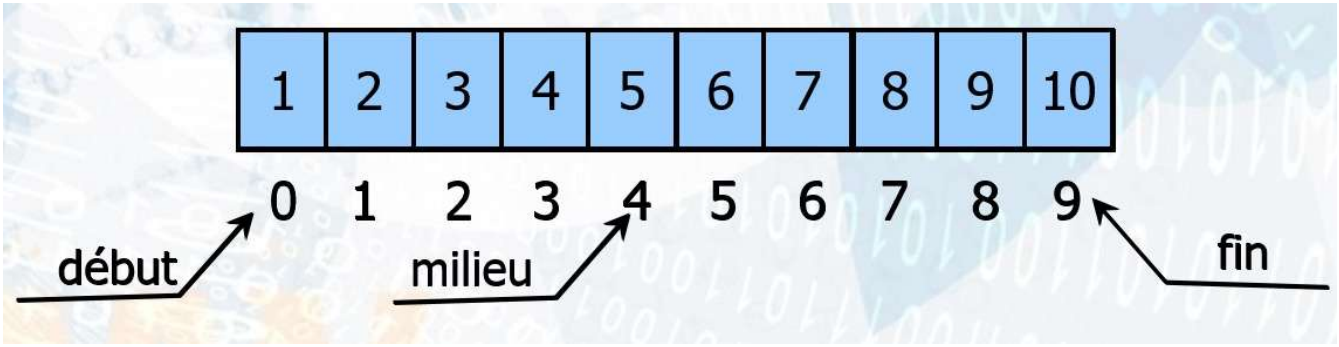
    if (je suis sortie parce que la premiere condition est fausse)
        return 1 ;   /// la valeur est presente
    else
        return 0 ;  /// la valeur n'est pas presente

}
```

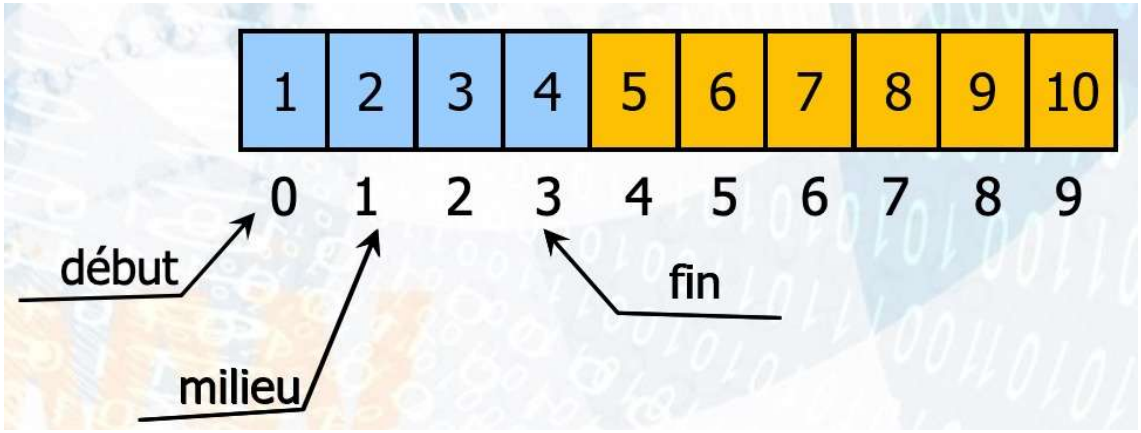
- Que se passe-t-il dans le pire des cas ?
  - Tableau d'1 million de valeurs, la Valeur cherchée est la dernière… 1 million de tests

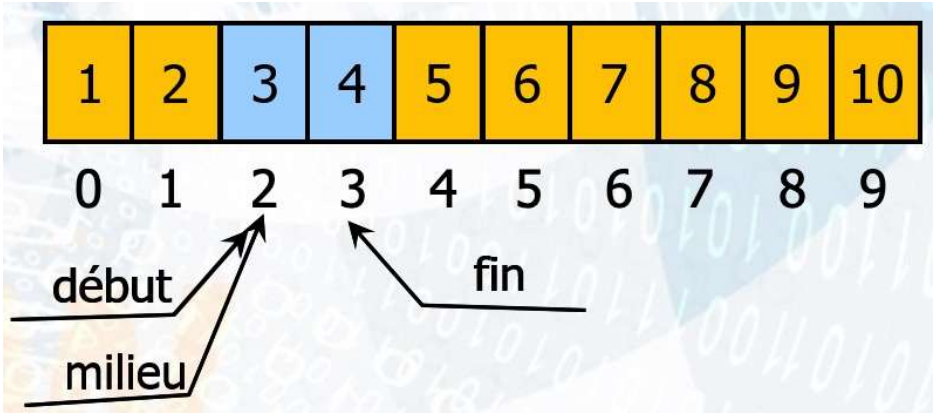
# Rechercher une valeur

- Un algo un peu plus évolué : recherche par dichotomie
  - Hypothèse importante : le tableau DOIT être trié
    - Exemple : on recherche la valeur 4



# Recherche dichotomique



# Recherche dichotomique

- Pseudo-code :

```
int gauche = 0;
int droite = tab.length - 1;
int milieu;
while (gauche <= droite) {
  milieu = (gauche + droite) / 2;
  if (x == tab[milieu])
    return milieu;
  if (x < tab[milieu])
    droite = milieu - 1;
  else
    gauche = milieu + 1;
  }
return -1;
```

# Recherche dichotomique

- Que se passe-t-il dans le pire des cas ?
  - Recherche d'une valeur parmi

| | |
|---|---|
| 1 000 000 | 976 |
| 500 000 | 488 |
| 250 000 | 244 |
| 125 000 | 122 |
| 62 500 | 61 |
| 31 250 | 30 |
| 15 625 | 15 |
| 7 812 | 8 |
| 3 906 | 4 |
| 1 953 | 2 |

# Trier un tableau

- Il existe beaucoup d'algos de tri, plus ou moins efficaces en terme de complexité
  - Nombre d'opérations effectuées (temps de calcul)
  - Tri par selection
  - Tri à bulle

# Tri par sélection

- On recherche le plus petit élément et on le place au début du tableau (ou le plus grand et on le place à la fin)
- On répète cette opération pour les N éléments du tableau
- On utilise une limite qui part du début du tableau et qui s'arrête à la fin
  - Cette limite sépare la partie triée du tableau de la partie non encore triée
  - Tous les éléments à gauche de cette limite sont triés et sont plus petits que les valeurs situées à droite de cette limite

# Tri par sélection

- Pseudo-code :

```
procédure tri_selection(tableau t)
    n ← longueur(t)
    pour i de 0 à n - 2
        min ← i
        pour j de i + 1 à n - 1
            si t[j] < t[min], alors min ← j
        fin pour
        si min ≠ i, alors échanger t[i] et t[min]
    fin pour
fin procédure
```

# Tri par sélection

| i | tab | boucle j |
|---|---|---|
| 0 | [3, 25, 50, 8, 1, 3, 49] | 1 2 3 4 5 6 |
| 1 | [1, 25, 50, 8, 3, 3, 49] | 2 3 4 5 6 |
| 2 | [1, 3, 50, 8, 25, 3, 49] | 3 4 5 6 |
| 3 | [1, 3, 3, 8, 25, 50, 49] | 4 5 6 |
| 4 | [1, 3, 3, 8, 25, 50, 49] | 5 6 |
| 5 | [1, 3, 3, 8, 25, 50, 49] | 6 |
| 6 | [1, 3, 3, 8, 25, 49, 50] | |

# Tri à bulle

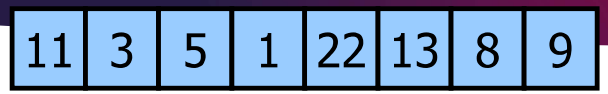
- Principe :
  - On parcourt le tableau et on compare 2 éléments successifs
  - Si 2 éléments successifs ne sont pas dans l'ordre croissant, on les échange
  - Lorsque l'on a parcouru tout le tableau sans faire d'échange, le tableau est trié et le programme s'arrête

# Tri à bulle

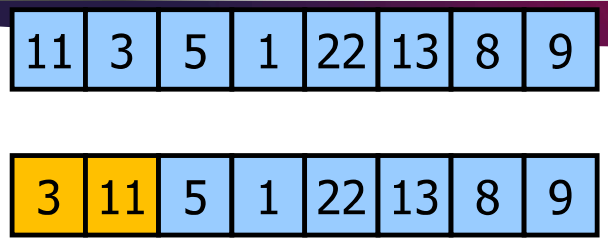
- Pseudo-code :

```
tri_à_bulles_optimisé(Tableau T)
    pour i allant de (taille de T)-1 à 1
        tableau_trié ← vrai
        pour j allant de 0 à i-1
            si T[j+1] < T[j]
                (T[j+1], T[j]) ← (T[j], T[j+1])
                tableau_trié ← faux
        si tableau_trié
            fin tri_à_bulles_optimisé
```

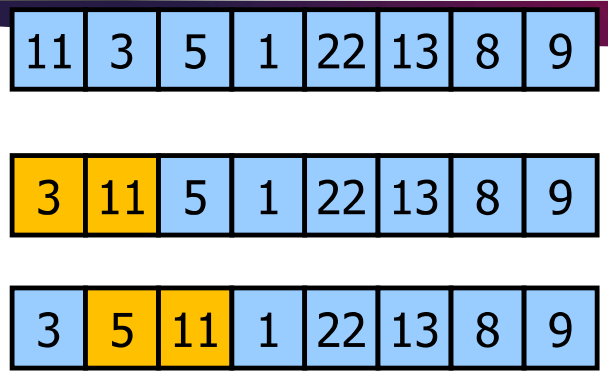
# Tri à bulle



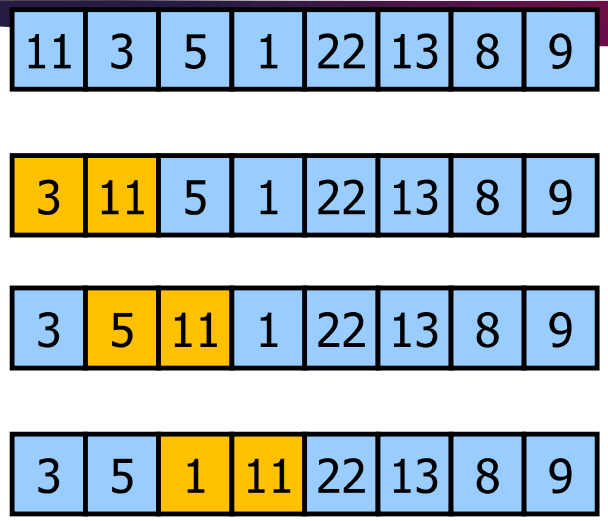
# Tri à bulle



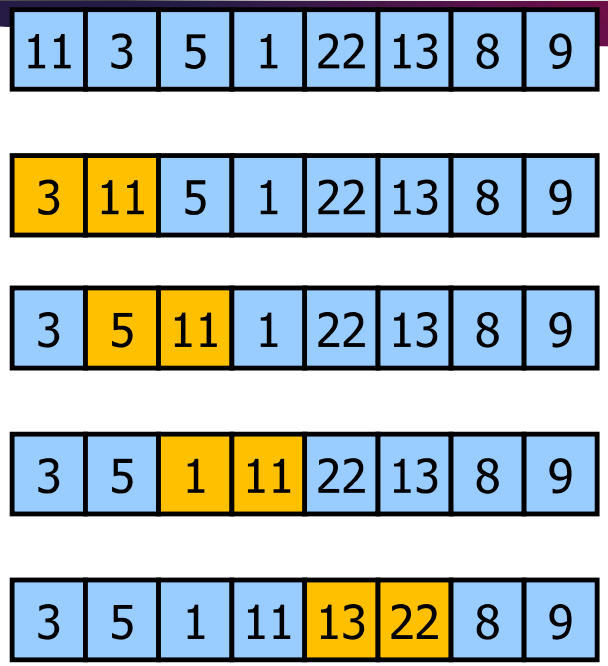
# Tri à bulle



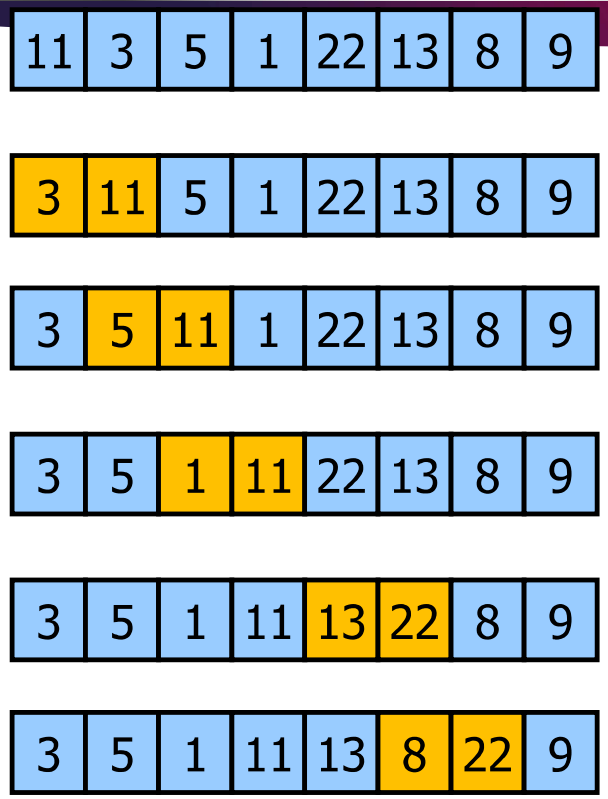
# Tri à bulle



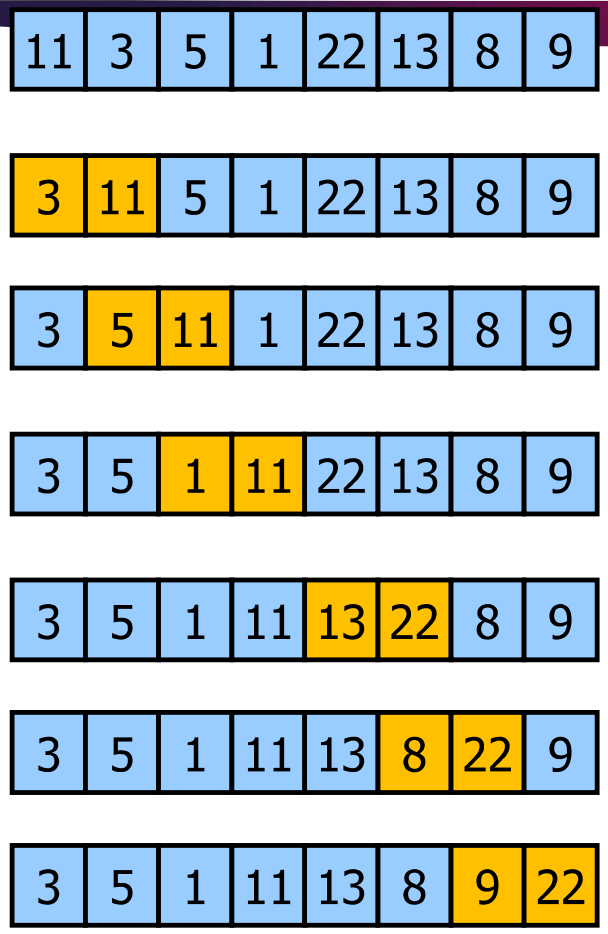
# Tri à bulle


11 3 5 1 22 13 8 9
3 11 5 1 22 13 8 9
3 5 11 1 22 13 8 9
3 5 1 11 22 13 8 9
3 5 1 11 13 22 8 9

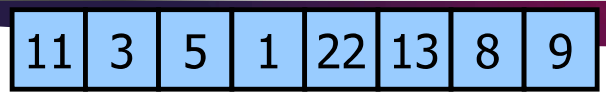
# Tri à bulle

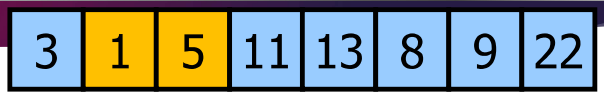

11 3 5 1 22 13 8 9
3 11 5 1 22 13 8 9
3 5 11 1 22 13 8 9
3 5 1 11 22 13 8 9
3 5 1 11 13 22 8 9
3 5 1 11 13 8 22 9

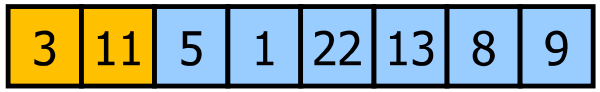
# Tri à bulle

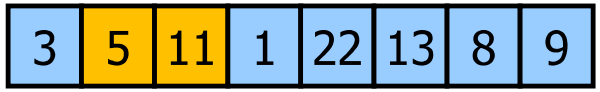

11 3 5 1 22 13 8 9
3 11 5 1 22 13 8 9
3 5 11 1 22 13 8 9
3 5 1 11 22 13 8 9
3 5 1 11 13 22 8 9
3 5 1 11 13 8 22 9
3 5 1 11 13 8 9 22

# Tri à bulle

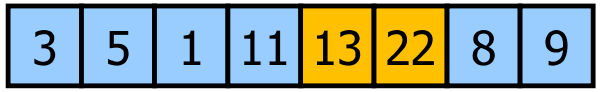

11 3 5 1 22 13 8 9

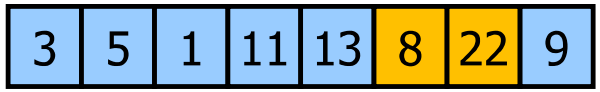

3 1 5 11 13 8 9 22

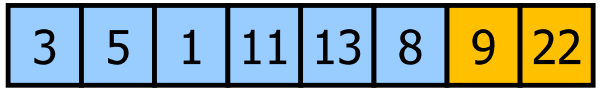

3 11 5 1 22 13 8 9


3 5 11 1 22 13 8 9

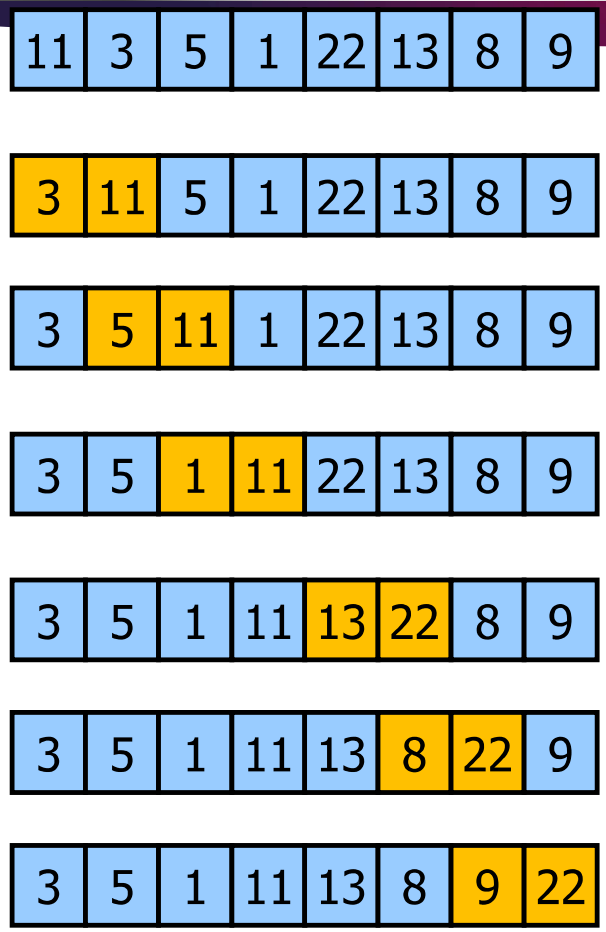

3 5 1 11 22 13 8 9

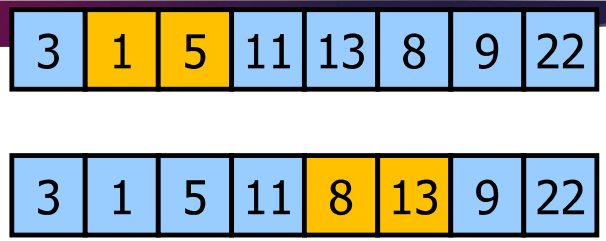

3 5 1 11 13 22 8 9


3 5 1 11 13 8 22 9

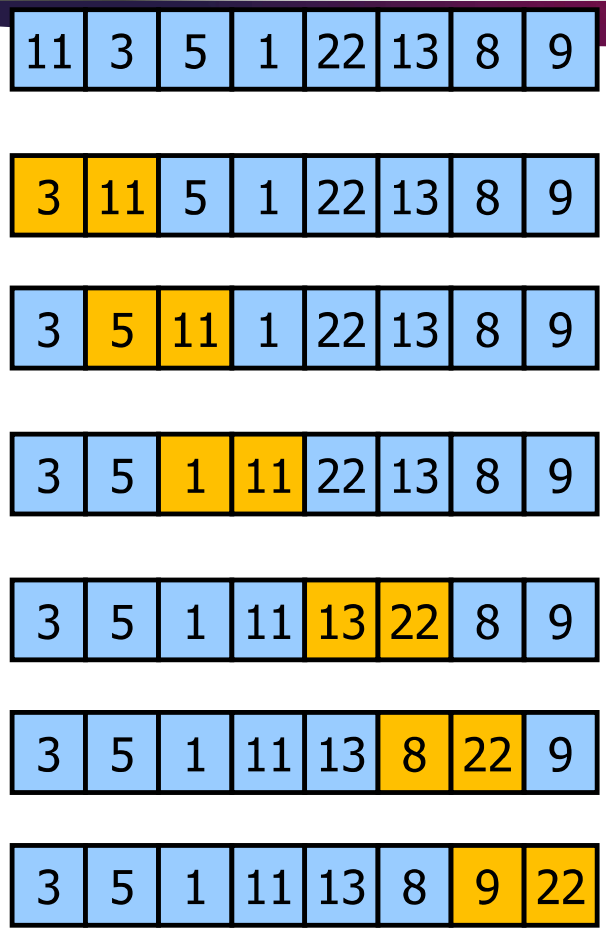

3 5 1 11 13 8 9 22

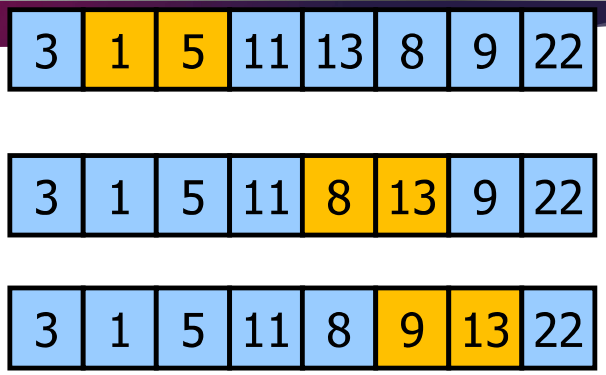
# Tri à bulle


11 3 5 1 22 13 8 9
3 11 5 1 22 13 8 9
3 5 11 1 22 13 8 9
3 5 1 11 22 13 8 9
3 5 1 11 13 22 8 9
3 5 1 11 13 8 22 9
3 5 1 11 13 8 9 22

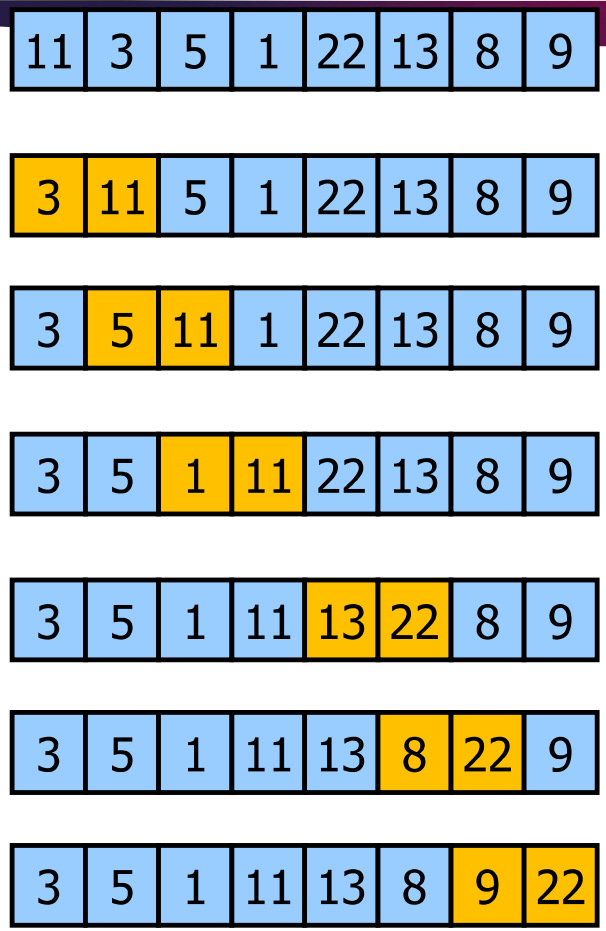

3 1 5 11 13 8 9 22
3 1 5 11 8 13 9 22

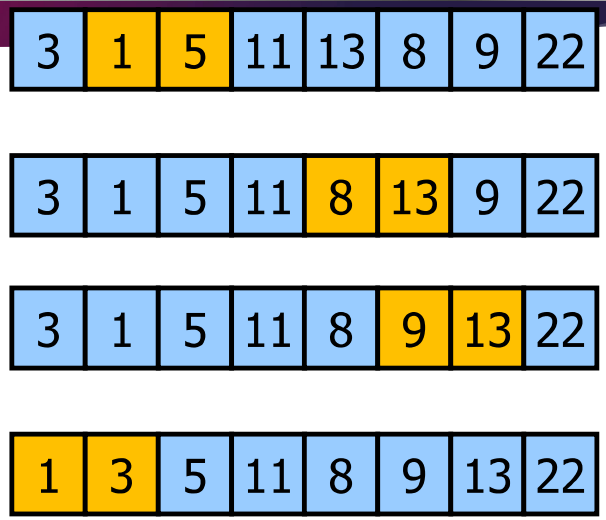
# Tri à bulle


11 3 5 1 22 13 8 9
3 11 5 1 22 13 8 9
3 5 11 1 22 13 8 9
3 5 1 11 22 13 8 9
3 5 1 11 13 22 8 9
3 5 1 11 13 8 22 9
3 5 1 11 13 8 9 22

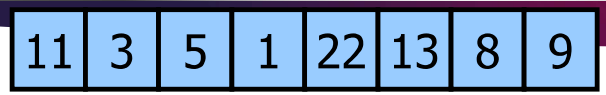

3 1 5 11 13 8 9 22
3 1 5 11 8 13 9 22
3 1 5 11 8 9 13 22

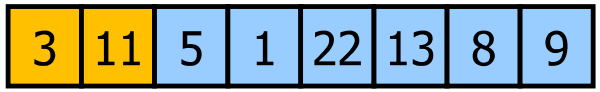
# Tri à bulle

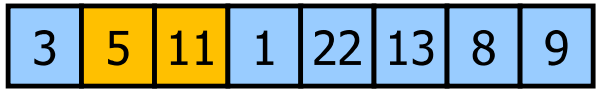

11 3 5 1 22 13 8 9
3 11 5 1 22 13 8 9
3 5 11 1 22 13 8 9
3 5 1 11 22 13 8 9
3 5 1 11 13 22 8 9
3 5 1 11 13 8 22 9
3 5 1 11 13 8 9 22


3 1 5 11 13 8 9 22
3 1 5 11 8 13 9 22
3 1 5 11 8 9 13 22
1 3 5 11 8 9 13 22

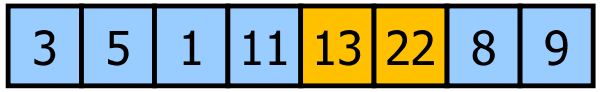
# Tri à bulle

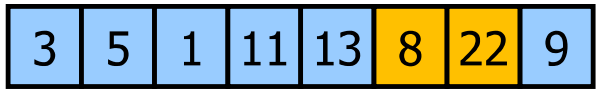

11 3 5 1 22 13 8 9

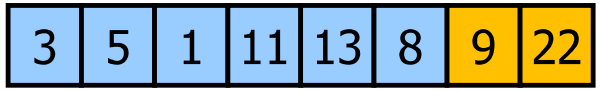

3 11 5 1 22 13 8 9

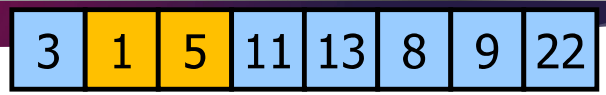

3 5 11 1 22 13 8 9


3 5 1 11 22 13 8 9

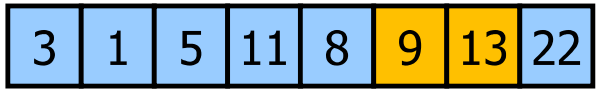

3 5 1 11 13 22 8 9

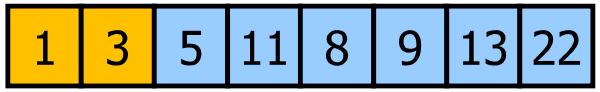

3 5 1 11 13 8 22 9


3 5 1 11 13 8 9 22


3 1 5 11 13 8 9 22

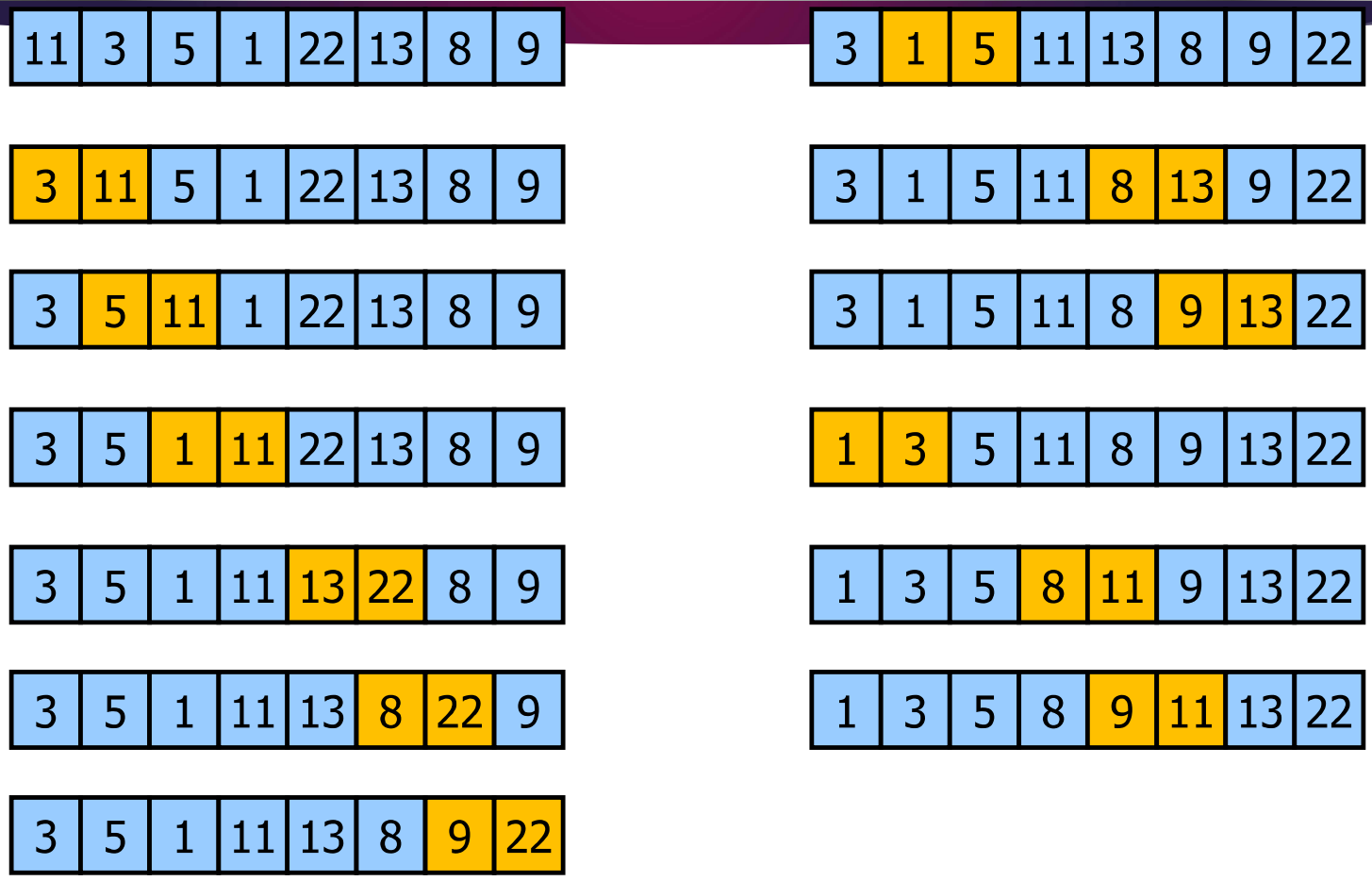

3 1 5 11 8 13 9 22


3 1 5 11 8 9 13 22


1 3 5 11 8 9 13 22


1 3 5 8 11 9 13 22

# Tri à bulle


11 3 5 1 22 13 8 9
3 11 5 1 22 13 8 9
3 5 11 1 22 13 8 9
3 5 1 11 22 13 8 9
3 5 1 11 13 22 8 9
3 5 1 11 13 8 22 9
3 5 1 11 13 8 9 22
3 1 5 11 13 8 9 22
3 1 5 11 8 13 9 22
3 1 5 11 8 9 13 22
1 3 5 11 8 9 13 22
1 3 5 8 11 9 13 22
1 3 5 8 9 11 13 22